

Unit IV: Logic and Inferences

Propositional logic in Artificial intelligence

Propositional logic (PL) is the simplest form of logic where all the statements are made by propositions. A proposition is a declarative statement which is either true or false. It is a technique of knowledge representation in logical and mathematical form.

Example:

1. a) It is Sunday.
2. b) The Sun rises from West (False proposition)
3. c) $3+3=7$ (False proposition)
4. d) 5 is a prime number.

Following are some basic facts about propositional logic:

- Propositional logic is also called Boolean logic as it works on 0 and 1.
- In propositional logic, we use symbolic variables to represent the logic, and we can use any symbol for representing a proposition, such as A, B, C, P, Q, R, etc.
- Propositions can be either true or false, but it cannot be both.
- Propositional logic consists of an object, relations or function, and **logical connectives**.
- These connectives are also called logical operators.
- The propositions and connectives are the basic elements of the propositional logic.
- Connectives can be said as a logical operator which connects two sentences.
- A proposition formula which is always true is called **tautology**, and it is also called a valid sentence.
- A proposition formula which is always false is called **Contradiction**.
- A proposition formula which has both true and false values is called **Contingency**.
- Statements which are questions, commands, or opinions are not propositions such as "Where is Rohini", "How are you", "What is your name", are not propositions.

Syntax of propositional logic:

The syntax of propositional logic defines the allowable sentences for the knowledge representation. There are two types of Propositions:

- a. **Atomic Propositions**

b. **Compound propositions**

- **Atomic Proposition:** Atomic propositions are the simple propositions. It consists of a single proposition symbol. These are the sentences which must be either true or false.

Example:

1. a) $2+2$ is 4, it is an atomic proposition as it is a **true** fact.
 2. b) "The Sun is cold" is also a proposition as it is a **false** fact.
- **Compound proposition:** Compound propositions are constructed by combining simpler or atomic propositions, using parenthesis and logical connectives.

Example:

1. a) "It is raining today, and street is wet."
2. b) "Ankit is a doctor, and his clinic is in Mumbai."

Logical Connectives:

Logical connectives are used to connect two simpler propositions or representing a sentence logically. We can create compound propositions with the help of logical connectives. There are mainly five connectives, which are given as follows:

1. **Negation:** A sentence such as $\neg P$ is called negation of P. A literal can be either Positive literal or negative literal.
2. **Conjunction:** A sentence which has $\wedge$ connective such as, $P \wedge Q$ is called a conjunction.
Example: Rohan is intelligent and hardworking. It can be written as,
P= Rohan is intelligent,
Q= Rohan is hardworking. $\rightarrow P \wedge Q$.

3. **Disjunction:** A sentence which has $\vee$ connective, such as $P \vee Q$. is called disjunction, where P and Q are the propositions.

Example: "Ritika is a doctor or Engineer",

Here P= Ritika is Doctor. Q= Ritika is Doctor, so we can write it as $P \vee Q$.

4. **Implication:** A sentence such as $P \rightarrow Q$, is called an implication. Implications are also known as if-then rules. It can be represented as

If it is raining, then the street is wet.

Let P= It is raining, and Q= Street is wet, so it is represented as $P \rightarrow Q$

5. **Biconditional:** A sentence such as $P \Leftrightarrow Q$ is a **Biconditional sentence**, example **If I am breathing, then I am alive**

P= I am breathing, Q= I am alive, it can be represented as $P \Leftrightarrow Q$.

Following is the summarized table for Propositional Logic Connectives:

Connective symbols	Word	Technical term	Example
$\wedge$	AND	Conjunction	$A \wedge B$
$\vee$	OR	Disjunction	$A \vee B$
$\rightarrow$	Implies	Implication	$A \rightarrow B$
$\leftrightarrow$	If and only if	Biconditional	$A \leftrightarrow B$
$\neg$ or $\sim$	Not	Negation	$\neg A$ or $\sim B$

Truth Table: In propositional logic, we need to know the truth values of propositions in all possible scenarios. We can combine all the possible combination with logical connectives, and the representation of these combinations in a tabular format is called **Truth table**. Following are the truth table for all logical connectives:

For Negation:

P	$\neg P$
True	False
False	True

For Conjunction:

P	Q	$P \wedge Q$
True	True	True
True	False	False
False	True	False
False	False	False

For disjunction:

P	Q	$P \vee Q$
True	True	True
False	True	True
True	False	True
False	False	False

For Implication:

P	Q	$P \rightarrow Q$
True	True	True
True	False	False
False	True	True
False	False	True

For Biconditional:

P	Q	$P \leftrightarrow Q$
True	True	True
True	False	False
False	True	False
False	False	True

Precedence of connectives:

Just like arithmetic operators, there is a precedence order for propositional connectors or logical operators. This order should be followed while evaluating a propositional problem. Following is the list of the precedence order for operators:

Precedence	Operators
First Precedence	Parenthesis
Second Precedence	Negation
Third Precedence	Conjunction(AND)
Fourth Precedence	Disjunction(OR)
Fifth Precedence	Implication
Six Precedence	Biconditional

Note: For better understanding use parenthesis to make sure of the correct interpretations. Such as $\neg R \vee Q$, It can be interpreted as $(\neg R) \vee Q$.

Logical equivalence:

Logical equivalence is one of the features of propositional logic. Two propositions are said to be logically equivalent if and only if the columns in the truth table are identical to each other.

Let's take two propositions A and B, so for logical equivalence, we can write it as $A \Leftrightarrow B$. In below truth table we can see that column for $\neg A \vee B$ and $A \rightarrow B$, are identical hence A is Equivalent to B

A	B	$\neg A$	$\neg A \vee B$	$A \rightarrow B$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Limitations of Propositional logic:

- We cannot represent relations like ALL, some, or none with propositional logic.
Example:

1. **All the girls are intelligent.**

1. Some apples are sweet.

- Propositional logic has limited expressive power.
- In propositional logic, we cannot describe statements in terms of their properties or logical relationships.

First-Order Logic in Artificial intelligence

In the topic of Propositional logic, we have seen that how to represent statements using propositional logic. But unfortunately, in propositional logic, we can only represent the facts, which are either true or false. PL is not sufficient to represent the complex sentences or natural language statements. The propositional logic has very limited expressive power. Consider the following sentence, which we cannot represent using PL logic.

- "Some humans are intelligent", or
- "Sachin likes cricket."

To represent the above statements, PL logic is not sufficient, so we required some more powerful logic, such as first-order logic.

First-Order logic:

- First-order logic is another way of knowledge representation in artificial intelligence. It is an extension to propositional logic.
- FOL is sufficiently expressive to represent the natural language statements in a concise way.
- First-order logic is also known as **Predicate logic or First-order predicate logic**. First-order logic is a powerful language that develops information about the objects in a more easy way and can also express the relationship between those objects.
- First-order logic (like natural language) does not only assume that the world contains facts like propositional logic but also assumes the following things in the world:
 - **Objects:** A, B, people, numbers, colors, wars, theories, squares, pits, wumpus,
 - **Relations:** It can be unary relation such as: red, round, is adjacent, or n-any relation such as: the sister of, brother of, has color, comes between
 - **Function:** Father of, best friend, third inning of, end of,
- As a natural language, first-order logic also has two main parts:
 - **Syntax**

- **Semantics**

Syntax of First-Order logic:

The syntax of FOL determines which collection of symbols is a logical expression in first-order logic. The basic syntactic elements of first-order logic are symbols. We write statements in short-hand notation in FOL.

Atomic sentences:

- Atomic sentences are the most basic sentences of first-order logic. These sentences are formed from a predicate symbol followed by a parenthesis with a sequence of terms.
- We can represent atomic sentences as **Predicate (term1, term2,, term n)**.

Example: Ravi and Ajay are brothers: $\Rightarrow$ Brothers(Ravi, Ajay).
Chinky is a cat: $\Rightarrow$ cat (Chinky).

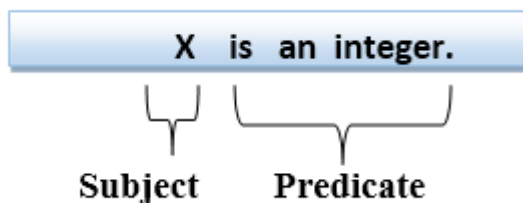
Complex Sentences:

- Complex sentences are made by combining atomic sentences using connectives.

First-order logic statements can be divided into two parts:

- **Subject:** Subject is the main part of the statement.
- **Predicate:** A predicate can be defined as a relation, which binds two atoms together in a statement.

Consider the statement: "x is an integer.", it consists of two parts, the first part x is the subject of the statement and second part "is an integer," is known as a predicate.



Quantifiers in First-order logic:

- A quantifier is a language element which generates quantification, and quantification specifies the quantity of specimen in the universe of discourse.

- These are the symbols that permit to determine or identify the range and scope of the variable in the logical expression. There are two types of quantifier:

1. **Universal Quantifier, (for all, everyone, everything)**

1. **Existential quantifier, (for some, at least one).**

Universal Quantifier:

Universal quantifier is a symbol of logical representation, which specifies that the statement within its range is true for everything or every instance of a particular thing.

The Universal quantifier is represented by a symbol $\forall$, which resembles an inverted A.

Note: In universal quantifier we use implication " $\rightarrow$ ".

If x is a variable, then $\forall x$ is read as:

- **For all x**
- **For each x**
- **For every x.**

Example:

All man drink coffee.

$\forall x \text{ man}(x) \rightarrow \text{drink}(x, \text{coffee}).$

It will be read as: There are all x where x is a man who drink coffee.

Existential Quantifier:

Existential quantifiers are the type of quantifiers, which express that the statement within its scope is true for at least one instance of something.

It is denoted by the logical operator $\exists$, which resembles as inverted E. When it is used with a predicate variable then it is called as an existential quantifier.

Note: In Existential quantifier we always use AND or Conjunction symbol ($\wedge$).

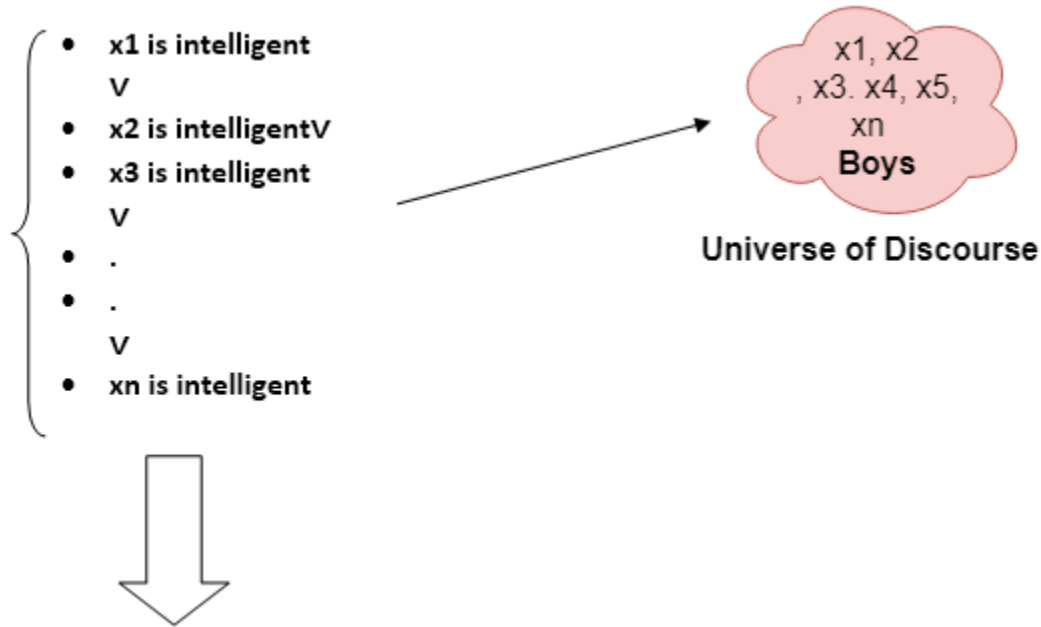
If x is a variable, then existential quantifier will be $\exists x$ or $\exists(x)$. And it will be read as:

- **There exists a 'x.'**
- **For some 'x.'**

- For at least one 'x.'

Example:

Some boys are intelligent.



So in short-hand notation, we can write it as:

$\exists x: \text{boys}(x) \wedge \text{intelligent}(x)$

It will be read as: There are some x where x is a boy who is intelligent.

Soundness and Completeness

Given a technique/system T that analyzes a program/input P and says either P has property X or P does not have property X,

- T is **sound**: if T says P has property X, then P does indeed have property X.

In other words, *T can be trusted when it says P has property X. T cannot be trusted when T says P does not have property X.*

Saying it another way, *T is truthful when it says P has property X. T may be untruthful when it says P does not have property X.*

- T is **complete**: if P has property X, then T will say P has property X.

In other words, *T cannot be trusted when it says P has property X. T can be trusted when T says P does not have property X.*

Saying it another way, *T may be untruthful when it says P has property X. T is truthful when it says P does not have property X.*

Let's consider two examples to understand these definitions.

Example: Security Analysis Tool

Consider a security analysis tool T that tells if the given program P leaks user information (vulnerability X). So, given a program P, T says either P has vulnerability X or P does not have vulnerability X. We will use T's verdict to advice folks to use or not use P.

1. If T was sound and said P has vulnerability X, then *users should not use P* cos' T can be trusted with this verdict — P is vulnerable with X.
2. If T was sound and said P does not have vulnerability X, then *users should further examine P before choosing to use it* cos' T cannot be trusted with this verdict — P may be vulnerable with X.
3. If T was complete and said P has vulnerability X, then *users should further examine P before choosing to use it* cos' T cannot be trusted with this verdict — P may not be vulnerable with X.
4. If T was complete and said P does not have vulnerability X, then *users should use P* cos' T can be trusted with this verdict — P is not vulnerable with X.

Example: May-Alias Analysis

Consider an alias analysis tool T that tells which pair of variables in a program P may be aliases (property X) at some (program) point during P's execution.

Unlike in this example, T can be perceived as giving a set of verdicts — one for every pair of variables in P. However, we consider each verdict separately, then this example is not different than the previous example.

In this example, the verdict is a bit fuzzy: T will say variables v1 and v2 *may be* aliases (property X) or T will say variables v1 and v2 *will/must not be* aliases (not property X). While this fuzziness does not affect how we deal with soundness and completeness of T, it affects the utility of the verdict when combined with the soundness and completeness of T.

1. T was sound and it said a variable pair V may be aliases (property X). Since we can trust a sound T when it says property X is true, variables in pair V may be aliases.
2. T was sound and it said a variable pair V' will/must not be aliases (not property X). Since we cannot trust a sound T when it says property X is false, it is possible that variables in V' may be aliases.
3. T was complete and it said a variable pair V may be aliases (property X). Since we cannot trust a complete T when it says property X is true, it is possible that variables in V' will not be aliases.
4. T was complete and it said a variable pair V' will/must not be aliases (not property X). Since we can trust a complete T when it says property X is false, variables in V' will not be aliases.

Observe that 1 and 2 together say nothing more than every variable pair in the program may be aliases. So, we gain no new information and, in this case, the soundness of T proves to be useless.

In contrast, 3 and 4 together helps us partition the set of variable pairs into two sets:

1. The set of variable pairs in which variables will not be aliases.
2. The set of variable pairs in which variables may be aliases.

So, we gain new information and, in this case, completeness of T proves to be useful.

Depending on the analysis and any fuzziness inherent in the verdict, the above observations will change. To see such changes, reason about the must variant of the second example — an alias analysis tool T that tells which pair of variables in a program P will/must be aliases (property X) at some (program) point during P's execution. You could also try reasoning about must-not variant and may-not variant of the second example.

Forward and Backward chaining

Forward Chaining and Backward Chaining are the two most important strategies in the field of [Artificial Intelligence](#) and lie in the Expert System Domain of AI.

Forward and Backward chaining is the strategies used by the Inference Engine in making the deductions.

Inference Engine:

Inference Engine is a component of the expert system that applies logical rules to the knowledge base to deduce new information. It interprets and evaluates the facts in the knowledge base in order to provide an answer.

A knowledgebase is a structured collection of facts about the system's domain.

Forward Chaining:

Forward Chaining the Inference Engine goes through all the facts, conditions and derivations before deducing the outcome i.e When based on available data a decision is taken then the process is called as Forwarding chaining, It works from an initial state and reaches to the goal(final decision).

Example:

A

A -> B

B

He is running.

If he is running, he sweats.

He is sweating.

Backward Chaining:

In this, the inference system knows the final decision or goal, this system starts from the

goal and works backwards to determine what facts must be asserted so that the goal can be achieved, i.e it works from goal(final decision) and reaches the initial state.

Example:

B

A -> B

A

He is sweating.

If he is running, he sweats.

He is running.

Difference between Forward Chaining and Backward Chaining:

	Forward Chaining	Backward Chaining
1.	When based on available data a decision is taken then the process is called as Forward chaining.	Backward chaining starts from the goal and works backward to determine what facts must be asserted so that the goal can be achieved.
2.	Forward chaining is known as data-driven technique because we reaches to the goal using the available data.	Backward chaining is known as goal-driven technique because we start from the goal and reaches the initial state in order to extract the facts.
3.	It is a bottom-up approach.	It is a top-down approach.
4.	It applies the Breadth-First Strategy.	It applies the Depth-First Strategy.
5.	Its goal is to get the conclusion.	Its goal is to get the possible facts or the required data.
6.	Slow as it has to use all the rules.	Fast as it has to use only a few rules.
7.	It operates in forward direction i.e it works from initial state to final decision.	It operates in backward direction i.e it works from goal to reach initial state.
8.	Forward chaining is used for the planning, monitoring, control, and interpretation application.	It is used in automated inference engines, theorem proofs, proof assistants and other artificial intelligence applications.

Forward search algorithms

Forward search algorithms are used to explore a search space starting from the initial state and moving towards the goal state. These algorithms systematically examine the reachable states from the current state and continue the process until the goal state is found or some termination condition is met. One of the most basic forward search algorithms is the Breadth-First Search (BFS) algorithm. Here is a description of the BFS algorithm:

Breadth-First Search (BFS):

BFS is a simple and widely used search algorithm that systematically explores a graph or tree by exploring all the neighbor nodes at the present depth before moving on to nodes at the next depth level. It guarantees the shortest path from the initial state to the goal state in an unweighted graph.

Here's the basic idea of the Breadth-First Search algorithm:

- ❖ Begin with the initial state as the current node.
- ❖ Enqueue the initial state into a queue data structure.
- ❖ Repeat the following steps until the queue is empty or the goal state is found:
- ❖ Dequeue a node from the queue and mark it as visited.
- ❖ Generate all possible neighboring states from the current node that have not been visited.
- ❖ Enqueue these newly generated states into the queue.
- ❖ If the goal state is found, the algorithm terminates, and the path to the goal can be reconstructed from the visited nodes.
- ❖ If the queue becomes empty before the goal state is found, there is no path from the initial state to the goal state.

BFS is particularly effective in finding the shortest path in unweighted graphs. However, it may not be efficient for large or complex state spaces due to its need to store all generated states in memory. Variants of BFS, such as Iterative Deepening Depth-First Search (IDDFS), are often used to overcome memory constraints and explore larger state spaces.

Backward search algorithms

Backward search algorithms, in contrast to forward search algorithms, start from the goal state and work backward towards the initial state. These algorithms are particularly useful when the goal state is well-defined and the system needs to determine the sequence of actions

or states that lead to the goal state. One common backward search algorithm is the Backward Chaining algorithm.

Backward Chaining:

Backward Chaining is an inference method commonly used in artificial intelligence and expert systems. It starts with the goal or a hypothesis and works backward to find any known facts or rules that support the goal. Backward Chaining is often employed in rule-based systems where the system attempts to find a sequence of actions or conditions that lead to a specific goal.

Here's a basic idea of the Backward Chaining algorithm:

- ❖ Begin with the goal state or the hypothesis that needs to be proven.
- ❖ Identify the rules or facts that support the goal and check if their conditions are satisfied.
- ❖ If the conditions are not satisfied, recursively apply the backward chaining algorithm to the subgoals until facts or rules are found that can be applied.
- ❖ Continue this process until either all subgoals are satisfied or a contradiction is reached.
- ❖ If the initial state is reached, the goal is proven, and the sequence of actions or states leading to the goal can be determined.

Backward Chaining is particularly useful in systems where the knowledge is organized in the form of rules or facts, and the system needs to reason backward to determine the sequence of actions leading to a specific goal. It is commonly used in the implementation of expert systems, diagnostic systems, and automated reasoning system